SALESFORCE DEVELOPER BOOTCAMP

DAY 5

Building Business Logic with Apex

A Complete Training Handbook — Placement Management System, Sprint 4

Version 1.0

Prepared By: Sudharshan Kuppili

Table of Contents

Chapter 1 — Introduction

Chapter 2 — Building Business Logic with Apex

Chapter 3 — Thinking Like an Architect

Chapter 4 — Discovering Apex

Chapter 5 — Engineering Sprint 1: Creating the Application Service

Chapter 6 — Engineering Sprint 2: Receiving an Application

Chapter 7 — Engineering Sprint 3: Preventing Duplicate Applications

Chapter 8 — Engineering Sprint 4: Validating Eligibility

Chapter 9 — Engineering Sprint 5: Saving Data

Chapter 10 — Engineering Sprint 6: Completing the Workflow

Chapter 11 — Debugging

Chapter 12 — Interview Corner

Chapter 13 — Sprint Retrospective

Chapter 14 — Complete Revision Notes

Chapter 15 — Glossary

Chapter 16 — Interview Cheat Sheet

Chapter 17 — Professional Best Practices

Preface — How to Use This Handbook

This handbook was assembled from four source documents that together make up the complete Day 5 training material for Sprint 4 of the Placement Management System project: "Building Business Logic with Apex," "Thinking Like an Architect," "Discovering Apex," and "Engineering Sprint 1" (which itself contains Engineering Sprints 1 through 6, a Debugging section, an Interview Corner, and a Sprint Retrospective).

The seventeen chapters that follow preserve the order of topics exactly as they appear across the four source documents, expanded in detail for a complete beginner. Every chapter ends with a set of callout boxes — Key Takeaways, Interview Notes, Common Mistakes, Best Practices, or a Revision Summary — so the same material can be studied in depth on a first read and revised quickly before an interview.

Two kinds of callout boxes are used to keep the origin of every explanation transparent, as requested for this handbook:

> **SOURCE-DERIVED CONTENT**
>
> The majority of this handbook — every business scenario, every code example, every diagram, every interview question, and every principle attributed to the Technical Lead — is drawn directly from the four uploaded documents. These sections are marked with a "Source:" citation naming the document and section they come from.

> **SUPPLEMENTARY EXPLANATION**
>
> A small number of boxes, clearly labelled "Beginner Analogy" or "Additional Explanation," add plain-language analogies or connect ideas across chapters to aid understanding. These boxes never introduce a new Salesforce feature, business rule, or engineering practice that is not already supported by the source material — they only restate or illustrate what the source material already establishes.

Read the chapters in order — each sprint and each chapter builds directly on the design decisions made in the one before it, exactly as the original material intended.

CHAPTER 1

Introduction

What This Sprint Is About

Sprint 4 of the Placement Management System project is the point where the application stops being a simple record-keeping tool and starts becoming genuine software. Over the preceding days, the development team built the foundation of the system: students, companies, job opportunities and applications could all be stored in Salesforce. Records could be created, viewed and updated, and the user interface looked professional.

At first glance, this looked like a finished product. But a Sprint Review with the Placement Officer revealed the truth: the application could store anything typed into it, but it could not tell the difference between a valid application and an invalid one. A student with six active backlogs had successfully applied to a company that explicitly required "No Active Backlogs." A student had submitted four applications for the same job. Applications had been accepted after the deadline had already passed. A company had accidentally created duplicate job postings.

> *"The system stores data, but it does not yet understand the business." — Technical Lead, Sprint 4 Review*

This single sentence is the reason Sprint 4 exists. Until now the team had been building a database — a place to store information. From this sprint onward, the team begins building software — a system capable of making correct decisions on its own, every time, without a human having to check each record by hand.

Why This Sprint Exists

Every piece of software that matters to a business exists because it needs to do more than remember information. A spreadsheet can remember information. What a spreadsheet cannot do is automatically refuse a late application, automatically detect that a student has already applied for the same job twice, or automatically check a student's CGPA against a company's minimum requirement before allowing the application to proceed.

Sprint 4 exists to close exactly this gap. Its purpose is to take the business rules that the Placement Office already follows in its head — rules nobody had ever written down as software — and turn them into logic the Salesforce application enforces automatically and consistently, every single time, for every single student.

What Business Problem Is Being Solved

The business problem is stated plainly by the Placement Officer in the Sprint Review: the application stores everything she enters, but it does not help her manage placements. Concretely, this sprint solves four business problems that had already caused real incidents:

- Students with active backlogs were able to apply to companies that explicitly disallow backlogs.
- A single student could submit multiple applications for the same job.
- Applications were being accepted after the stated deadline had passed.
- Duplicate company/job records could be created by mistake, with nothing stopping it.

Each of these is not a coding problem in isolation — it is a business rule that has not yet been given a home inside the software. Sprint 4 is where these rules find that home.

> **BEGINNER ANALOGY (Supplementary Explanation)**
>
> Think of the Placement Management System, before Sprint 4, as a filing cabinet with excellent labels. You can put any paper in any folder, and the cabinet will hold it neatly. But the cabinet has no opinion about whether the paper belongs there. It will happily file a rejected application as if it were accepted, because a filing cabinet does not "think" — it only stores. Sprint 4 is the moment the filing cabinet gets a brain: a clerk who checks each paper before filing it and rejects the ones that break the rules.

Learning Objectives

By the end of this sprint, a developer working through this material should be able to:

- Understand why enterprise applications require business logic.
- Distinguish between data and business decisions.
- Identify business rules from customer requirements.
- Design Apex classes based on business responsibilities.
- Explain why good engineers spend time understanding problems before writing code.
- Think like a consultant while analysing requirements.
- Prepare to implement business logic using Apex.

Source: Chapter 4 — Building Business Logic with Apex, "Learning Outcomes"

Engineering Principles Introduced in This Sprint

This sprint is built around one governing idea, repeated by the Technical Lead in different forms throughout all four source documents:

> *"Understand the business first. Write the code afterwards."*

Every other principle in this sprint is really a restatement of this one idea, applied to a different situation:

- Every line of code should answer the question: "Which business problem does this solve?"
- Professional software engineers think about architecture before implementation.
- A component should have one primary responsibility.
- Classes exist because businesses have responsibilities that must be organised, not because Apex requires them.
- Build small, test often, improve continuously — do not attempt to solve every problem in one large step.

Together, these principles form the engineering mindset the rest of this handbook will apply, chapter by chapter, to the Placement Management System.

> **KEY TAKEAWAYS**
>
> Sprint 4 marks the shift from a system that stores data to a system that understands the business.
>
> The business problem is that the application accepts invalid applications because no business rules are

enforced in software.

The governing engineering principle for the entire sprint is: understand the business first, write the code afterwards.

INTERVIEW NOTES

If asked "why do enterprise applications need business logic, not just a database?", the strongest answer is the Placement Office story itself: a database will happily store an invalid application; only business logic can refuse to save it in the first place.

Interviewers often listen for the phrase "business decision" versus "data storage" — being able to draw this line clearly is a strong signal of engineering maturity.

REVISION SUMMARY

Sprint 4 gives intelligence to the Placement Management System.

The business problem: the system stores data but does not enforce business rules.

The core principle: business understanding comes before code.

CHAPTER 2

Building Business Logic with Apex

What Is Business Logic?

Business logic is the intelligence that allows software to behave according to the needs of an organisation. It is the set of rules and decisions a system applies automatically, so that people do not have to manually check every record by hand.

Consider a simple calculator. You enter two numbers, and it adds them. The calculator follows a rule, but it is a fixed, mechanical rule — it has no concept of "business." Now consider the Placement Management System. A student applies for a company. Should the application always be accepted? Certainly not. The software must first answer a series of questions before it can decide:

- Does the student satisfy the minimum CGPA requirement?
- Does the student have active backlogs?
- Has the application deadline expired?
- Has the student already applied for this job?
- Is the student already holding another offer that prevents further applications?
- Has the company restricted applications to certain branches?

Only after evaluating these conditions should the software decide whether to accept or reject the application. This decision-making ability — the software choosing an outcome based on rules, rather than blindly saving whatever it receives — is business logic.

Source: Chapter 4, Section 4.1 — "What Makes Software Intelligent?"

Why Business Logic Exists

Without business logic, software becomes nothing more than an electronic register. It stores information faithfully, but it does not assist people in making decisions, and it cannot protect the business from mistakes. This is precisely what happened to the Placement Management System before Sprint 4 — every field was captured correctly, but nothing checked whether the data made sense together.

Business logic exists to close that gap: to make sure the software actively enforces the rules the organisation already lives by, instead of passively recording whatever is typed into it.

The Difference Between CRUD and Business Logic

CRUD stands for Create, Read, Update, and Delete — the four basic operations any database-backed system performs on records. Before Sprint 4, the Placement Management System could already do all four: it could create a student record, read it back, update it, and delete it. This is exactly the capability the Technical Lead is describing when he says "we have been building a database."

Business logic is a different layer entirely. It sits in front of, or alongside, CRUD operations and decides whether an operation should be allowed to happen at all, and under what conditions. CRUD asks "can this data be stored?" Business logic asks "should this data be stored, given everything we know about the business rules?"

ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)

This distinction — CRUD versus business logic — is not spelled out with that exact terminology in the source documents, but it is the natural vocabulary for the difference the Technical Lead draws between "storing data" and "understanding the business." CRUD is the mechanical act of saving or fetching a record. Business logic is the judgment applied before or after that mechanical act.

Why Software Should Make Decisions

A reasonable question a beginner might ask is: why should the software make these decisions automatically? Couldn't the Placement Officer simply review every application by hand and reject the invalid ones herself?

The source material's own evidence answers this question. The Placement Officer's reports show four separate failures that slipped past manual oversight — an ineligible student, a duplicate application, a late submission, and a duplicate job posting. If a human reviewer, whose full-time job is placements, can miss these problems, the volume and repetition of the checks make them exactly the kind of work software should do: the same rule, applied identically, every single time, without fatigue or oversight.

BEGINNER ANALOGY (Supplementary Explanation)

This is the same reason an ATM checks your balance before letting you withdraw cash, instead of trusting a bank employee to remember every customer's balance from memory. The rule ("do not allow withdrawal greater than the available balance") is simple, but it must be applied perfectly, every time, for every customer — which is precisely what software is good at and humans, over thousands of repetitions, are not.

Enterprise Examples of Business Rules

Every organisation operates according to a well-defined, if often unwritten, set of rules. Schools have attendance rules. Banks have transaction rules. Hospitals have patient care rules. Universities have examination rules. The Placement Office is no different — it too operates according to rules that, until Sprint 4, existed only in the Placement Officer's head and not in the software.

Business Requirement	Business Rule
Students should not apply after the deadline.	Reject late applications.
Duplicate applications are not allowed.	Check existing applications before saving.
Companies specify eligibility criteria.	Validate CGPA, branch and backlog requirements.
Recruiters should know when eligible students apply.	Send notifications after successful submission.
Placement Officers should maintain accurate data.	Prevent duplicate company records where appropriate.

Source: Chapter 4, Section 4.2 — "Understanding Business Rules"

Notice that none of these statements mention Apex, SOQL, or any Salesforce feature at all. They simply describe how the business wants the software to behave. Only after these rules are understood can developers decide how to implement them — which is exactly why professional software engineers spend considerable time discussing requirements before writing a single line of code.

The Placement Office Example, in Full

The Placement Office example is the anchor story for this entire sprint, and it is worth walking through slowly because every later chapter refers back to it.

The Placement Officer arrives at the Sprint Review carrying printed reports. She opens the first: a student with six active backlogs applied to a company that explicitly stated "No Active Backlogs" as a requirement, and the application was accepted anyway. She opens another: a single student submitted four separate applications for the same job. Another: applications were accepted even after the stated deadline had already passed. A final one: a company accidentally created duplicate job postings, and nothing in the system stopped it.

Each of these examples maps directly onto one of the business rules in the table above — CGPA/backlog eligibility, duplicate-application prevention, deadline enforcement, and duplicate-record prevention. This is not a coincidence: the reports are the evidence that these rules were needed, and the table is the translation of that evidence into requirements a developer can act on.

Company Examples

The "company accidentally created duplicate job postings" incident is a useful second example because it shows that business logic protects the business on both sides of the transaction — not only the student applying, but also the company posting the job. Left unchecked, duplicate job postings would confuse students (which listing is the real one?), pollute reporting, and make the Placement Office look unprofessional to recruiting partners. The same principle — validate before saving — applies whether the record being validated is a Student, a Job, or an Application.

Real-World Salesforce Examples

Within the Salesforce platform specifically, this same pattern of "software making decisions" is what the Technical Lead is preparing the team to build using Apex, SOQL, and DML, described later in the source material as the tools that let a business service:

- represent a business service as an Apex class,
- represent a business activity as an Apex method,
- represent business information as Apex variables,
- represent groups of business records as collections,
- retrieve information needed for decision-making using SOQL, and
- store the results of those decisions using DML.

Source: Section 4.7 — "Preparing for Apex"

This list is the bridge between the business-first thinking of this chapter and the hands-on Apex work that begins in Chapter 4.

> **KEY TAKEAWAYS**

Business logic is the intelligence that lets software make correct decisions, not just store data.

CRUD operations (Create, Read, Update, Delete) are the mechanical actions; business logic decides whether those actions should happen at all.

Every organisation — schools, banks, hospitals, the Placement Office — already has rules; Sprint 4 is about translating those existing rules into software.

The Placement Officer's four reported incidents map directly onto four concrete business rules the team must now implement.

INTERVIEW NOTES

A strong interview answer to "what is business logic?" contrasts it directly with CRUD: CRUD is the mechanism, business logic is the judgment applied on top of the mechanism.

Be ready to give a concrete example, ideally one you can explain in one sentence — e.g., "reject an application if the student's CGPA is below the company's minimum requirement" is business logic; "save the application record" is plain CRUD.

COMMON MISTAKES

Treating every requirement as a coding problem instead of first asking "what decision is the business trying to make here?"

Assuming that because the user interface looks complete and polished, the underlying application is also complete — as the Placement Office example shows, a good-looking UI can sit on top of an application with zero business logic.

REVISION SUMMARY

Business logic = the decision-making layer of software, distinct from CRUD.

Every business rule starts as an unwritten habit inside an organisation before it becomes software.

The Placement Office's four incidents are the concrete evidence behind Sprint 4's four core business rules.

CHAPTER 3

Thinking Like an Architect

> *"Every successful software application is built twice — first in the mind of the engineer, and then in the programming language."*

From Requirements to Design

By this point the team understands the business problem: the Placement Office wants software that can make decisions. Several business rules have already been identified — validate eligibility, prevent duplicate applications, enforce application deadlines, notify recruiters, and maintain accurate records.

The next question is an important one: where should each of these responsibilities be implemented?

Many beginner developers immediately begin writing code wherever they find it convenient. Some place business logic inside Lightning Web Components. Some write everything inside one large Apex class. Some copy the same code into multiple places. Initially, everything appears to work. After a few weeks, however, every small change becomes difficult — the application becomes harder to understand, harder to test, and harder to maintain.

Professional software engineers avoid this situation by thinking about architecture before implementation.

Software Architecture

Architecture is simply the organised arrangement of responsibilities within a software system. Just as a well-designed building places bedrooms, kitchens and staircases in appropriate locations, a well-designed application assigns each responsibility to the most suitable component. Good architecture makes software easier to understand, easier to modify and easier to extend.

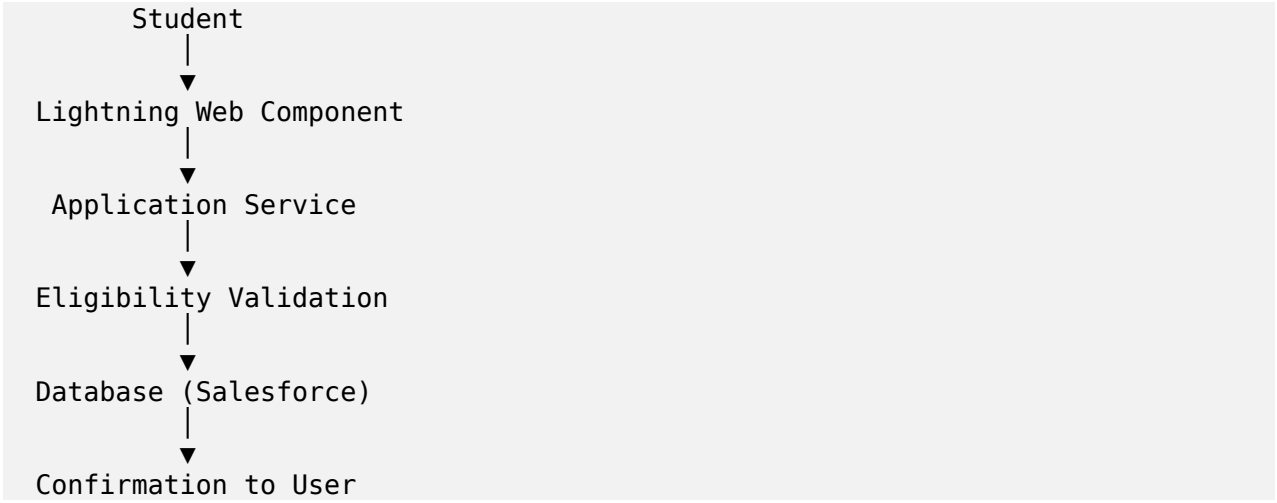
> **BEGINNER ANALOGY (Supplementary Explanation)**
>
> Imagine trying to build a house where the kitchen sink, the bedroom, and the electrical wiring are all crammed into one room with no walls between them. Technically the house might still function, but the moment you need to fix the plumbing, you risk damaging the wiring. Software without architecture is that house — everything works at first, but nothing can be changed safely.

Responsibilities

Imagine a student submits an application for a job. Many things happen almost simultaneously: the application is received, eligibility is verified, the record is saved, a notification may be sent, and reports may later be updated. Although these actions occur together in time, they do not all belong to the same software component. Each component has its own responsibility.

Thinking in this way — asking "whose job is this, really?" for every action a system performs — is one of the most important habits of professional software engineers.

The Journey of a Student Application



The user interface receives information. The business service makes decisions. The database stores information. Each layer performs a different responsibility, and this separation keeps the application organised.

Source: Sections 4.4–4.5 — "Thinking Like an Architect" and "Every Component Has a Responsibility"

Separation of Concerns

The diagram above is a direct illustration of a principle known industry-wide as separation of concerns: each part of a system should be responsible for one concern (one category of problem) and should not reach into the concerns owned by another part. The Lightning Web Component's concern is the user interface — accepting input and displaying output. The Application Service's concern is business decision-making. The database's concern is durable storage.

> **ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)**
>
> The term "separation of concerns" itself is not used verbatim in the source documents, but it is the standard industry name for exactly the pattern the source material describes with the phrase "each component has its own responsibility." Naming it explicitly is useful for interviews, where this exact term is frequently asked about.

Single Responsibility Principle

> *A component should have one primary responsibility.*

When every component performs one responsibility well, the entire application becomes easier to understand. Whenever a component starts doing too many unrelated tasks, it becomes difficult to maintain. Professional developers constantly ask themselves: "Is this responsibility in the right place?"

This idea reappears, described from a different angle, in the discussion of Apex classes: "One Responsibility. One Service." A class, a method, and a whole layer of an application are all expected to obey the same rule — pick one job, and do only that job.

Layered Architecture: UI Layer, Business Layer, Database Layer

The "Journey of a Student Application" diagram is, in effect, a three-layer architecture:

Layer	Component	Responsibility
UI Layer	Lightning Web Component	Accept user input. Display output.
Business Layer	ApplicationService	Business decisions. Validation. Processing.
Database Layer	Salesforce Database	Store records. Retrieve records.

Source: Section 4.5, "Every Component Has a Responsibility"

> **BEGINNER ANALOGY (Supplementary Explanation)**
>
> This three-layer split mirrors a restaurant. The waiter (UI Layer) takes your order and brings you your food — they do not decide whether the kitchen is out of an ingredient. The kitchen (Business Layer) decides what can be cooked, in what order, and whether a substitution is allowed. The store room (Database Layer) simply holds the ingredients and does not decide anything at all — it just keeps what it is given, safely, until it's needed.

Designing the Application Services

The Technical Lead gathers the team around the whiteboard and writes three class names: StudentService, JobService, ApplicationService. He asks: "What should each class do?"

StudentService

This service performs operations related to students:

- Registering students.
- Updating student profiles.
- Verifying academic information.
- Checking placement status.

Notice that this class should not create job postings or process applications — those responsibilities belong elsewhere.

JobService

This service is responsible for everything related to job opportunities:

- Creating new job postings.
- Updating eligibility criteria.
- Closing expired opportunities.
- Publishing available jobs.

Again, this class should not process student applications.

ApplicationService

This service becomes the heart of the sprint. It is responsible for the application process itself, including:

- Receiving applications.
- Checking eligibility.
- Preventing duplicate applications.
- Saving successful applications.
- Returning meaningful messages to the user.

> *"Notice that we have designed almost the entire solution." — "But we haven't written any Apex yet." — "Exactly. Good design always comes before implementation."*

Source: Section 4.6 — "Designing the Application Services"

Pod Activity — Think Like an Engineer

The Placement Office introduces a new requirement: "Students who already hold two offers cannot apply for any additional companies." Working through this as a design exercise (before writing any code) is instructive:

- Which service should implement this rule? — This is a rule about whether an application should be accepted, so it belongs inside ApplicationService, most likely as part of the eligibility checks it already performs.
- Should this rule be checked before or after saving the application? — Before saving, so that an ineligible application is never written to the database in the first place.
- Will this change affect StudentService, JobService or ApplicationService? — Primarily ApplicationService; StudentService may need to expose the student's current offer count, but the decision itself stays in ApplicationService.
- If the rule changes from two offers to three next year, which part of the application needs to change? — Only the single validation check inside ApplicationService, because the rule was never duplicated elsewhere.

Source: Section 4.6, "Think Like an Engineer" pod activity

> **TEAM LEAD TIP**
>
> "Many new developers believe that architecture slows development. In reality, architecture prevents confusion. A few minutes spent organising responsibilities can save weeks of rework later. Before writing your first line of Apex today, ask yourself one final question: Do I understand the business well enough to explain my solution without mentioning code? If your answer is yes, you are ready to begin programming."

Preparing for Apex

With the business problem, the business rules, the responsibilities of each service, and the overall architecture now understood, only one step remains: a programming language capable of expressing these responsibilities as executable software. Within the Salesforce platform, that language is Apex.

Apex allows developers to translate business rules into software that executes automatically and consistently. Concretely:

- An Apex class can represent a business service.

- An Apex method can represent a business activity.
- Apex variables can represent business information.
- Collections can represent groups of business records.
- SOQL retrieves information required for decision-making.
- DML stores the results of those decisions.

Technology follows understanding. It never replaces it. This is exactly how enterprise software projects evolve — the business is understood first, and the technology is chosen and applied afterward because it fits the design, not the other way around.

Source: Section 4.7 — "Preparing for Apex"

KEY TAKEAWAYS

Architecture is the organised arrangement of responsibilities within a system, decided before implementation.

The Journey of a Student Application shows three layers: UI (LWC), Business (ApplicationService), and Database — each with one job.

Single Responsibility Principle: one component, one primary responsibility.

StudentService, JobService and ApplicationService each own a distinct, non-overlapping set of responsibilities.

Apex is chosen only after the design is understood — technology follows understanding, it never replaces it.

INTERVIEW NOTES

"What is separation of concerns?" — Each part of the system owns one category of responsibility (UI, business logic, or data) and does not reach into another part's job.

"What is the Single Responsibility Principle?" — A class or component should have one, and only one, reason to change.

Be ready to redraw the three-layer diagram (UI → Business → Database) from memory and explain what each layer is and is not allowed to do.

COMMON MISTAKES

Placing business logic inside a Lightning Web Component instead of a dedicated Apex service class.

Writing one large Apex class that tries to handle students, jobs, and applications all together.

Copying the same validation logic into multiple places instead of giving it one home.

BEST PRACTICES

Design the responsibilities on a whiteboard (or paper) before opening an IDE.

Ask "is this responsibility in the right place?" whenever a class or method starts to feel crowded.

Name services after the business capability they represent (StudentService, JobService,

ApplicationService), not after technical actions.

REVISION SUMMARY

Architecture = organised responsibility, decided before code.

Three layers: UI receives input, Business Service decides, Database stores.

Three services: StudentService, JobService, ApplicationService — one responsibility each.

Apex is the language chosen to implement a design that already exists in the team's mind.

CHAPTER 4

Discovering Apex

> *"Programming is nothing more than expressing good design in a language that computers can understand."*

From Design to Code

Every discussion up to this point has focused on the business: understanding the Placement Office, identifying business rules, organising responsibilities, and designing services. Now comes the question: how do we convert these ideas into software? The answer is Apex.

Many beginners think Apex is something entirely new that must be memorised from scratch. Professional developers see it differently — they see Apex as a way of expressing ideas they have already understood. The business comes first; the programming language simply gives life to the design. This is why experienced developers often begin writing code much faster than beginners: they are not discovering the solution while coding, they discovered it during the design discussion. Coding is simply the final translation.

Source: Section 4.8 — "Discovering Apex"

Your First Apex Class, Explained Line by Line

Of the three responsibilities identified earlier — register students, publish jobs, process applications — the team agrees to automate processing applications first. The Technical Lead writes the first class:

```
public class ApplicationService {

}
```

"This is not merely an Apex class," he says. "It is the software representation of one business responsibility." Let's take this apart word by word, assuming no prior programming knowledge at all.

public

The word public is an access modifier. It tells Salesforce that this class is accessible from other classes in the org. Without going further into the different levels of access Apex supports, the beginner-level understanding is: public means other pieces of code, elsewhere in the application, are allowed to use this class.

class

The keyword class tells Apex that what follows is the definition of a class — a named container that groups together related pieces of behaviour. Declaring a class does not run any code by itself; it simply creates the container that other code will later be placed inside.

ApplicationService (naming)

ApplicationService is the name chosen for this class, and the choice of name is deliberate, not decorative. It is a business-focused class name — it tells any reader, instantly, what business responsibility this class represents (processing applications), rather than describing a technical detail. Good naming conventions in

Apex, as throughout the source material, favour names that describe business responsibility over names that describe implementation.

{ } — Braces

The curly braces { } mark the beginning and end of the class body. Everything written between these two braces belongs to the class — it is the boundary of the container. An empty pair of braces, as in this first version, means the class exists but currently has no behaviour inside it yet.

Why Classes Exist

That single sentence — "it is the software representation of one business responsibility" — changes the way a class should be thought of. An Apex class is not just a container for code. It is a carefully organised place where one business capability is implemented.

Whenever a developer creates a new class, the first question should never be "what code will I write?" Instead, ask: "which business responsibility does this class represent?"

> **BEGINNER ANALOGY (Supplementary Explanation)**
>
> Imagine a hospital. Doctors diagnose patients. Pharmacists dispense medicines. Receptionists register patients. Each person has a different responsibility. Now imagine asking one person to perform all three jobs — the hospital would quickly become chaotic. Software behaves in exactly the same way: every service should focus on one well-defined responsibility. When responsibilities remain clear, software becomes easier to understand, easier to test and easier to modify.

> *Engineering Principle: One Responsibility. One Service.*

Source: Sections 4.9, "Your First Apex Class," and the hospital analogy that follows it

Business Responsibility and Enterprise Design

This same reasoning — is there a new business responsibility that deserves its own place? — is the test a developer should apply whenever they are unsure whether a new class is required. If the answer is yes, a new class is probably needed. If the answer is no, the design should be reconsidered rather than adding an unrelated method to an existing class.

Giving the Service Something to Do

A class without methods is like a company without employees. It exists, but nothing happens. The Technical Lead asks: "What should ApplicationService actually do?" Students immediately suggest ideas: accept an application, check eligibility, save the application, display confirmation, reject duplicate applications.

These are not methods yet — they are business activities. Each activity can eventually become an Apex method. The Technical Lead writes the first one:

```
public class ApplicationService {

    public void submitApplication(){

    }
```

```
}
```

Notice that the method is named after a business activity — not a technical activity. Good method names tell a story: someone reading the code should understand what the software is doing without reading every statement inside the method.

Source: Section 4.10 — "Giving the Service Something to Do"

Pod Activity — Identifying Methods

Suppose the Placement Officer asks for additional features: withdraw an application, approve an application, reject an application, view application history, reopen an application. Without writing any Apex, the exercise is to identify which of these should probably become methods inside ApplicationService, and then to discuss — with another pod — whether every one of them really belongs in this service, or whether some of them might belong elsewhere (for example, an approval workflow might eventually need its own service if it grows complex enough).

Source: Section 4.10, "Think Like an Engineer" pod activity

Understanding Parameters

The next question the Technical Lead asks is: "when a student submits an application, what information should the software receive?" The class quickly answers: Student Id, and Job Id. Without this information, the service cannot determine who is applying or for which job. Therefore, the method should receive these values as parameters.

```
public class ApplicationService {

    public void submitApplication(Id studentId,
                                  Id jobId){

    }

}
```

Parameters are simply pieces of information supplied to a method so that it can perform its responsibility. Every parameter should answer one question: what information does the business activity require? Avoid passing unnecessary information — simple methods are usually easier to understand and maintain.

Source: Section 4.11 — "Understanding Parameters"

> **ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)**
>
> For a complete beginner: Id is simply the data type used here, meaning "this parameter holds a Salesforce record identifier." studentId and jobId are the parameter names — chosen, just like the method name, to describe the business information they represent rather than anything technical.

> **TEAM LEAD TIP**
>
> Good software engineers think about method names before they think about parameters. They think about parameters before they think about implementation. They think about implementation before

> they think about optimisation. Changing this order often leads to confusion.

Returning Results

Imagine the student clicks the Apply button. The software performs several checks. What should happen next? Should the user always see "Application Submitted Successfully"? Certainly not — the software should communicate the actual outcome. Examples include:

- Application submitted successfully.
- CGPA requirement not satisfied.
- Application deadline has expired.
- Duplicate application detected.
- Student already holds maximum permitted offers.

Every business activity produces a result, and methods should communicate those results clearly. Initially, this may simply be a success or failure message; later, as the application becomes more sophisticated, methods may return more detailed information. The important lesson: software should never leave users wondering what happened. Clear communication is part of good software design.

Source: Section 4.12 — "Returning Results"

> **TEAM LEAD TIP**
>
> "Never write methods that surprise other developers. A good method should clearly communicate three things: what it does, what information it needs, and what result it produces. If another developer understands those three things, they can usually use your code correctly without reading its implementation."

Building Before Expanding

Many beginners become excited after learning their first few Apex concepts and immediately begin writing large classes containing hundreds of lines of code. Professional engineers follow a different approach: they build small, they test often, they improve continuously.

The first version of a class or method does not need to solve every possible business problem — it only needs to solve today's problem correctly. As new requirements arrive, the design evolves. This gradual evolution is one of the defining characteristics of successful enterprise software. Software development is not about producing the largest amount of code; it is about producing software that continues to remain useful as the business grows.

Source: Section 4.13 — "Building Before Expanding"

Sprint Checkpoint

Before continuing to the implementation exercises, a pod should be able to confidently answer the following, without looking at the code:

- Why does ApplicationService exist?
- What business responsibility does it represent?

- Why do we create methods?
- How do method names reflect business activities?
- Why are parameters required?
- What should methods communicate after completing their work?

Source: Section 4.13, "Sprint Checkpoint"

KEY TAKEAWAYS

public class ApplicationService { } — public means accessible elsewhere, class defines the container, ApplicationService is a business-focused name, and the braces mark the class body.

A class represents one business responsibility; a method represents one business activity.

Method names should be named after business activities (submitApplication), not technical actions.

Parameters are the information a method needs to do its job — nothing more.

Every method should clearly communicate its outcome; never leave the user guessing what happened.

Build small, test often — do not write the entire class in one uninterrupted burst.

INTERVIEW NOTES

Be ready to explain public class ApplicationService { } word by word, assuming the interviewer wants to test fundamentals, not just usage.

"Why is the method called submitApplication() and not process()?" — because good names describe business activity, and someone reading the code should understand its purpose without opening the implementation.

"What should a method return when it fails?" — a clear, specific message describing what went wrong (e.g. "CGPA requirement not satisfied"), not a generic failure message.

COMMON MISTAKES

Naming a class or method after a technical action instead of a business responsibility.

Writing a large method that tries to do everything at once instead of building it incrementally.

Returning a generic "something went wrong" message instead of a specific, meaningful one.

Passing more parameters into a method than the business activity actually requires.

REVISION SUMMARY

public class ApplicationService { } is the software representation of the "process applications" responsibility.

public void submitApplication(Id studentId, Id jobId) adds a method representing the business activity of receiving an application, with exactly the parameters it needs.

Every method must eventually communicate a clear result — success or a specific reason for failure.

Build the smallest version that solves today's problem; expand only as new requirements arrive.

CHAPTER 5

Engineering Sprint 1

Creating the Application Service

Sprint Context

You are now a member of the Salesforce Development Team at VIT Solutions Pvt. Ltd. Your client, the Placement Office, has approved the system design and is eager to see the first working version of the application. Today's sprint has one objective: implement the business logic that allows students to apply for jobs correctly. The team is no longer discussing ideas — it is now building software.

Sprint Backlog

Story ID	User Story	Priority
US-1	Accept a student application.	High
US-2	Prevent duplicate applications.	High
US-3	Check application deadline.	High
US-4	Verify student eligibility.	High
US-5	Save the application successfully.	High
US-6	Display meaningful feedback to the user.	Medium

The team commits to completing every story in sequence, and does not move to the next story until the previous one has been reviewed by another pod member.

Source: Part IV — "Engineering Sprint," Sprint Objective and Sprint Backlog

---

Business Requirement

The Placement Office wants one dedicated service responsible for handling all application-related operations.

Engineering Thinking

Before writing any code, the sprint asks the team to discuss three questions:

- Why should application processing be separated from student registration?
- If another developer joins the project six months later, where would they expect application logic to exist?
- Would creating multiple unrelated responsibilities inside one class make future maintenance easier or harder?

These questions are meant to be discussed for five minutes before opening an IDE — the sprint deliberately delays coding until the reasoning is clear.

Business Scenario

This sprint continues directly from Chapter 3's design discussion, where the team already agreed that ApplicationService should exist as the class responsible for the application process. Sprint 1 is the moment that design decision is finally written into the codebase.

Architecture Thinking

Separating application processing from student registration follows directly from the Single Responsibility Principle introduced in Chapter 3: StudentService owns student-related operations, and ApplicationService owns application-related operations. If a new developer joins the project later, a class named ApplicationService immediately tells them where application logic lives — they do not need to search through StudentService or a Lightning Web Component to find it. Mixing unrelated responsibilities into one class would make this discovery process much harder, and would make future changes riskier, since a change intended for one responsibility could accidentally affect another.

Problem Analysis

At this stage, the class should only contain the structure required to support future business operations. The task explicitly instructs: do not worry about implementation yet — focus on creating a clean foundation.

Step-by-Step Solution

- Step 1 — Decide the class name: ApplicationService, matching the business responsibility it represents.
- Step 2 — Declare the class as public, so it can be used by other parts of the application (such as the Lightning Web Component that will later call it).
- Step 3 — Leave the body of the class empty for now, since no business activity has been implemented yet.

Implementation

```
public class ApplicationService {

}
```

Expected Output

By the end of this exercise, the project should contain a dedicated service that clearly represents one business responsibility — nothing more, and nothing less.

Professional Best Practices

- Name the class after the business responsibility it represents, not after a technical detail.
- Keep the class empty until there is an actual business activity to add — do not add placeholder methods "just in case."

- Treat class creation as a design decision, not a formality.

Common Mistakes

- Adding application-processing logic directly into StudentService because "it's already there."
- Naming the class something generic like Helper or Utils instead of ApplicationService.
- Adding methods before the team has agreed on what the class is responsible for.

Interview Questions

- Why create an empty class before adding any methods to it?
- How would you explain the purpose of ApplicationService to a non-technical stakeholder?

Peer Review Checklist

- Appropriate class name.
- Clear business responsibility.
- No unnecessary code.
- Good formatting and readability.

Source: Part IV, "Engineering Sprint 1 — Creating the Application Service"

Revision Notes

- ApplicationService is created as an empty public class before any business activity is added.
- Its existence alone communicates a design decision: application processing is separated from student and job processing.

Summary

Sprint 1 turns the design decision from Chapter 3 into a real, if still empty, Apex class. The class's name and its emptiness are both intentional: the name signals its responsibility, and the emptiness signals that no business activity has been added until the team is ready to add it deliberately, one capability at a time.

> **REVISION SUMMARY**
>
> ApplicationService begins as an empty public class.
>
> Its sole purpose at this stage is to establish a clean foundation for the application-processing responsibility.
>
> Separation from StudentService and JobService follows the Single Responsibility Principle.

CHAPTER 6

Engineering Sprint 2

Receiving an Application

Business Requirement

A student selects a company and clicks Apply. The software should receive the request and begin processing it.

Engineering Thinking — What Minimum Information Is Needed?

Before coding, the pod discusses what minimum information the software should receive. The discussion points raised are:

- Student Identifier
- Job Identifier
- Date of Application

The sprint deliberately asks a second, sharper question: should anything else be required, and can unnecessary information create confusion? This connects directly back to the parameter-design guidance from Chapter 4 — every parameter should answer the question "what information does this business activity require?", and nothing more.

Method and Parameters, Explained

The coding task is to create a method named submitApplication(), accepting the required parameters. Chapter 4 already introduced this exact method signature while explaining parameters in general; Sprint 2 is where it is actually built as part of the running sprint.

```
public class ApplicationService {

    public void submitApplication(Id studentId,
                                  Id jobId){

        // Step 1: acknowledge that the request was received.
        System.debug('Application request received.');

    }

}
```

void — the method does not hand back a value to whatever called it (in this early version, its job is simply to receive the request, not yet to return a detailed result).

submitApplication — the business activity being represented: a student submitting an application.

studentId — identifies which student is applying.

jobId — identifies which job they are applying for.

Source: Part IV, "Engineering Sprint 2 — Receiving an Application"; method structure per Section 4.11

Business Activities Represented

At this stage, the instruction is explicit: simply display a success message indicating that the request has been received. Do not perform validations yet — build incrementally. This is a direct application of the "build small, test often" principle from Chapter 4: Sprint 2's only job is to prove the method can receive a request at all, before any business rule is layered on top of it.

Expected Behaviour

The software successfully receives an application request. No business validation has been added yet — that begins in the next sprint.

> **REMEMBER**
>
> Engineering Principle: Build one capability at a time. Professional developers avoid solving five problems simultaneously.

Professional Best Practices

- Accept only the parameters the business activity genuinely requires (studentId, jobId).
- Resist the temptation to add validation logic in the same step as receiving the request.
- Confirm receipt with a simple, visible signal before adding any complexity.

Common Mistakes

- Adding eligibility or duplicate checks in the same sprint as simply receiving the request, instead of building incrementally.
- Accepting extra parameters that are not yet needed, "in case they're useful later."

Interview Questions

- Why does submitApplication() start as a method that only acknowledges receipt, with no validation at all?
- What risk is avoided by building one capability at a time rather than all capabilities together?

> **REVISION SUMMARY**
>
> submitApplication(Id studentId, Id jobId) is the method that receives a student's application request.
>
> At this stage it performs no validation — it only confirms that the request was received.
>
> The guiding principle: build one capability at a time.

CHAPTER 7

Engineering Sprint 3

Preventing Duplicate Applications

Business Problem

A student should not be allowed to apply for the same company more than once. This is the exact problem the Placement Officer raised in Chapter 1: "a student submitted four applications for the same job."

Duplicate Validation — Think Like a Consultant

Before writing Apex, the sprint poses three questions: how can software determine whether the application already exists? Where should this validation occur — before saving or after saving? And why?

The answer that follows from the rest of the source material is: the check must happen before saving, using SOQL to search for an existing record. If the check happened after saving, a duplicate record would already exist in the database by the time it was detected, which is exactly the failure the Placement Officer reported.

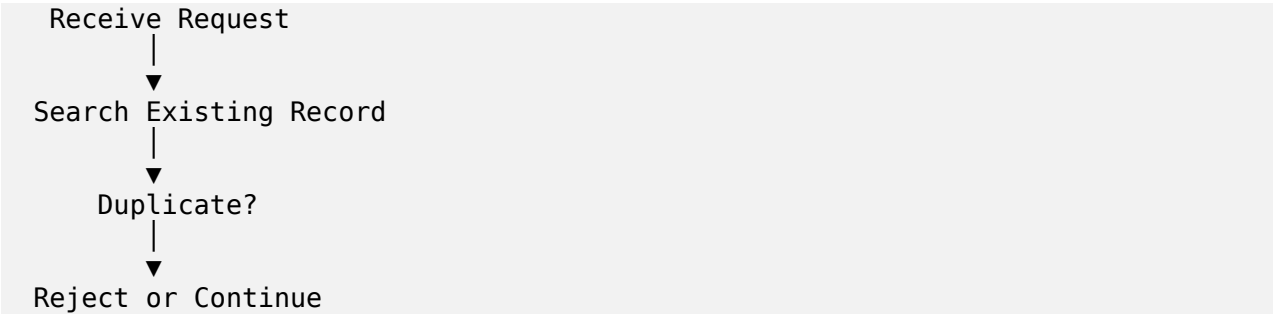
Why Before Saving?

Validating before saving means the invalid record is never written to Salesforce at all — there is nothing to clean up afterward, and no window of time in which the duplicate exists. This is consistent with the design decision made in Chapter 3's pod activity for the "maximum two offers" rule, where the conclusion was also that eligibility should be checked before the record is saved.

Introducing SOQL

SOQL (Salesforce Object Query Language) is the tool used to answer the question "does an application already exist for this student and this job?" Chapter 3 already previewed this role for SOQL when it said SOQL retrieves information required for decision-making — Sprint 3 is the first sprint where that retrieval actually happens, to support the duplicate check.

Flow Diagram



Source: Part IV, "Engineering Sprint 3 — Preventing Duplicate Applications"

Coding Task

Modify submitApplication() so that it first checks whether an application already exists. If a duplicate is found, display an appropriate message. Otherwise, continue processing.

```
public class ApplicationService {

    public void submitApplication(Id studentId,
                                  Id jobId){

        // Step 1: search for an existing application
        //         for this student and this job.
        // Step 2: if found, reject with a clear message.
        // Step 3: otherwise, continue processing.

    }

}
```

> **ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)**
>
> The source material describes the required behaviour of this step precisely (search, then reject-or-continue) but does not print the literal SOQL statement or the full method body in these four documents. The comments above represent that described behaviour as pseudocode, faithful to the described steps, without inventing a specific query syntax the source did not provide.

Real-World Examples

This same pattern — search before you save, to avoid a duplicate — is exactly what happens when a bank checks whether an account already exists for a given ID number before opening a new one, or when an airline checks whether a passenger already holds a seat on a flight before issuing a second boarding pass.

Test Scenarios

Scenario	Expected Result
First application	Accepted
Second application for same job	Rejected
Same student applying for another company	Accepted

Source: Part IV, "Engineering Sprint 3," Test Scenarios table

The third scenario is worth pausing on: the rule is not "a student may only ever submit one application." It is narrower and more precise — a student may not apply twice to the same job. Applying to a different company is still allowed. Getting this distinction right is exactly why the "Think Like a Consultant" discussion happens before any code is written.

Best Practices / Professional Approach

- Check for duplicates before saving, never after.
- Scope the duplicate check precisely — same student and same job, not merely the same student.
- Keep the validation logic simple enough that another developer can understand it within one minute.

Peer Review

The sprint's own review question is: can another developer understand your validation logic within one minute? If not, simplify it.

Common Mistakes

- Checking for duplicates after the record has already been saved.
- Rejecting all applications from a student who has applied elsewhere, instead of only duplicate applications for the same job.

Interview Questions

- Why must duplicate checking happen before the DML save, not after?
- What SOQL-based approach would you use to check whether a record already exists for a given student and job combination?

> **REVISION SUMMARY**
>
> Duplicate applications are prevented by searching for an existing record before saving, using SOQL.
>
> The rule is scoped to "same student, same job" — not "same student, any job."
>
> Flow: Receive Request → Search Existing Record → Duplicate? → Reject or Continue.

CHAPTER 8

Engineering Sprint 4

Eligibility Validation

Business Requirement

Companies specify eligibility criteria, and only eligible students should be allowed to apply. The checks required are:

- CGPA
- Active Backlogs
- Branch
- Graduation Year

Each of these checks maps directly back to the business rules identified in Chapter 2 and to the very first incident the Placement Officer reported: a student with six active backlogs was accepted by a company requiring none.

Why Helper Methods Are Used

Before coding, the sprint asks: which validations belong together? Should all validation logic remain inside one large method, or would smaller helper methods improve readability? The professional design that follows from this discussion breaks submitApplication() into a coordinating method that calls a series of smaller, single-purpose helper methods:

```
submitApplication()
  |-- checkDuplicate()
  |-- validateCGPA()
  |-- validateBacklogs()
  |-- validateBranch()
  |-- validateGraduationYear()
  |-- saveApplication()
```

Source: Part IV, "Engineering Sprint 4 — Validating Eligibility," Professional Design

Explaining the Design

This structure is the Single Responsibility Principle (Chapter 3) applied at the method level, not just the class level. Each helper method — validateCGPA(), validateBacklogs(), validateBranch(), validateGraduationYear() — has exactly one job, matching the naming guidance from Chapter 4: a good method name tells a story, and a reader should understand what each of these five checks does without reading a single line inside them.

submitApplication() itself becomes a coordinator: it does not perform the checks directly, it simply calls each helper method in order, in the same way ApplicationService itself coordinates responsibilities without performing StudentService's or JobService's jobs.

Clean Architecture

This layered breakdown — one coordinating method delegating to several small, focused helper methods — is what "clean architecture" means in practice at the method level. It is the same idea introduced in Chapter 3's three-layer diagram (UI → Business → Database), scaled down to fit inside a single class: submitApplication() is the coordinator, and each validate*() method is a specialist with one job.

Maintainability

Because each rule lives in its own helper method, changing one rule does not risk breaking another. If the minimum CGPA requirement changes, only validateCGPA() needs to be touched — validateBacklogs(), validateBranch(), and validateGraduationYear() are untouched and cannot be accidentally broken by the change.

Scalability — The Challenge Question

The sprint poses a deliberately open challenge: how would your design change if every company had different eligibility rules? This is a genuine architectural question, and the material invites discussion rather than providing a single fixed answer. What the existing design does make clear is the direction such a change would take: because each rule already lives in its own helper method rather than being tangled together, a design that needs to vary rules per company would extend this same pattern — for example, by making each validate*() method aware of the specific company's criteria — rather than requiring the whole method to be rewritten from scratch.

> **ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)**
>
> The source material poses the "different rules per company" challenge as a discussion question and does not provide or imply a specific implementation. The explanation above stays at the level the source supports — noting why the existing helper-method structure makes such a change more manageable — without inventing a concrete mechanism (such as a specific custom object or configuration approach) that the source text does not describe.

Coding Task

Extend submitApplication() to validate the required eligibility conditions, returning meaningful messages whenever a condition is not satisfied — directly reusing the "Returning Results" guidance from Chapter 4 (Section 4.12): "CGPA requirement not satisfied," not a generic failure.

> **BEST PRACTICES**
>
> Give each eligibility rule its own helper method.
>
> Name each helper method after the business rule it checks (validateCGPA, not checkOne).
>
> Return a specific, meaningful message identifying which rule failed.

> **COMMON MISTAKES**
>
> Cramming all four eligibility checks into a single block of logic inside submitApplication() itself.

Returning the same generic rejection message regardless of which rule actually failed.

INTERVIEW NOTES

"Why break eligibility validation into separate helper methods instead of one big method?" — readability, testability, and the ability to change one rule without risking the others.

"How would you design for eligibility rules that differ by company?" — acknowledge it as an open architectural question, and explain how the existing per-rule helper-method structure is a reasonable foundation to extend.

REVISION SUMMARY

Eligibility validation covers CGPA, Backlogs, Branch, and Graduation Year.

submitApplication() becomes a coordinator that calls checkDuplicate(), four validate*() helper methods, and saveApplication().

Breaking validation into helper methods improves readability, maintainability, and scalability.

CHAPTER 9

Engineering Sprint 5

Saving the Application

Business Requirement

Once every validation succeeds, the application should be stored in Salesforce. This is the final, decisive step in the whole workflow — everything up to this point (receiving the request, checking for duplicates, validating eligibility) exists to make sure that only a correct, legitimate application reaches this step.

Introducing DML

DML (Data Manipulation Language) is the Salesforce tool used to insert, update, delete, or upsert records in the database. Chapter 3 already introduced its role in the architecture: DML stores the results of the decisions made by the business logic. Sprint 5 is where that storage actually happens.

```
insert application;
```

Purpose: store the record in Salesforce, permanently, only once every prior check has passed.

Source: Part IV, "Engineering Sprint 5 — Saving the Application"

Error Handling

The sprint raises a direct question: what happens if Salesforce cannot save the record? Should the user simply see "Something went wrong," or should the software provide a more meaningful explanation? The material's own answer is unambiguous: good software communicates clearly.

Meaningful Feedback

This requirement connects directly back to Section 4.12's "Returning Results" guidance: every business activity produces a result, and methods should communicate those results clearly. A save failure is still a result — it deserves the same clarity as a successful save or a validation rejection, not a vague catch-all message.

Professional Design

Handled this way, saveApplication() takes its place as the final step in the coordinated design introduced in Sprint 4:

```
submitApplication()
  |-- checkDuplicate()
  |-- validateCGPA()
  |-- validateBacklogs()
  |-- validateBranch()
  |-- validateGraduationYear()
  |-- saveApplication()   <-- DML happens here, last
```

Notice that the DML statement is deliberately the very last thing that happens. Every other step exists to guarantee that, by the time execution reaches insert application;, the record is already known to be valid.

BEST PRACTICES

Perform the DML operation only after every validation has already passed.

Handle the possibility that a save can fail, and respond with a specific, informative message.

Keep saveApplication() focused only on the act of saving — validation belongs in the helper methods that ran before it.

COMMON MISTAKES

Attempting to save a record before all validations have completed.

Showing the user a generic "something went wrong" message instead of explaining what actually happened.

INTERVIEW NOTES

"Why does the DML statement come last in this design?" — because everything before it exists to guarantee the record is valid by the time it is saved.

"What is DML?" — the Salesforce operations (insert, update, delete, upsert) used to change what is stored in the database.

REVISION SUMMARY

DML = Insert, Update, Delete, Upsert — the operations that change stored data.

insert application; saves the record only after every prior check has passed.

Save failures deserve the same clear communication as any other result.

CHAPTER 10

Engineering Sprint 6

Completing the Workflow

The Complete Sequence

At this point, the software should perform the following sequence, in order:

```
Receive Application
        │
        ▼
 Validate Duplicate
        │
        ▼
Validate Eligibility
        │
        ▼
   Save Record
        │
        ▼
  Display Result
```

> *"If your implementation follows this sequence, you have successfully built your first business service. Congratulations."*

Source: Part IV, "Engineering Sprint 6 — Completing the Workflow"

Explaining Every Step

- Receive Application — submitApplication(studentId, jobId) is called; the request enters the system (Sprint 2).
- Validate Duplicate — checkDuplicate() searches for an existing application using SOQL, before anything is saved (Sprint 3).
- Validate Eligibility — validateCGPA(), validateBacklogs(), validateBranch(), and validateGraduationYear() each check one business rule (Sprint 4).
- Save Record — saveApplication() runs the DML insert, only once every prior check has passed (Sprint 5).
- Display Result — a clear, specific message is returned to the user, whether the outcome is success or a named reason for rejection (Section 4.12).

How Professional Salesforce Applications Work Internally

This six-step sequence is, in miniature, exactly what the three-layer architecture from Chapter 3 looks like in motion: the Lightning Web Component triggers Receive Application, the ApplicationService performs Validate Duplicate and Validate Eligibility, the Salesforce Database performs Save Record, and the result travels back up through the same layers to become the Display Result the student actually sees.

This is also a direct, working instance of the "Journey of a Student Application" diagram from Chapter 3 — Student → LWC → ApplicationService → Eligibility Validation → Database → Confirmation Message — now filled in with the specific methods built across Sprints 1 through 5.

KEY TAKEAWAYS

The complete workflow is: Receive → Validate Duplicate → Validate Eligibility → Save → Display Result.

Each step in this workflow corresponds to one sprint and one method (or group of helper methods) built earlier.

This sequence is the concrete, working realisation of the architecture designed in Chapter 3.

INTERVIEW NOTES

Be ready to draw this six-step workflow from memory and name which method (or group of methods) implements each step.

"Why does validation happen before saving, in every single case?" — so that an invalid record is never written to the database in the first place.

REVISION SUMMARY

Receive Application → Validate Duplicate → Validate Eligibility → Save Record → Display Result.

This is the finished shape of the first business service built in this handbook.

It mirrors, exactly, the architecture diagram introduced back in Chapter 3.

CHAPTER 11

Debugging

Every software engineer spends a significant part of the day debugging. Learning to recognise poor code is as important as learning to write good code. The source material presents four situations under the heading "Debug This!" Each is expanded below with an explanation of why it is wrong and what the correct approach looks like.

Source: Part IV, "Debug This!"

Situation 1 — A SOQL Query Written Inside a Loop

The scenario: a SOQL query is written inside a loop. The discussion question posed is: why might this become a problem when hundreds of students apply simultaneously, and how can the code be improved?

Why It Is Wrong

If a query runs once per loop iteration, then processing one student costs one query, but processing two hundred students costs two hundred queries — the cost scales directly with the number of records being processed. Salesforce enforces strict limits on how many queries a single transaction may perform. A pattern that works fine with a handful of test records can fail outright the first time it meets a realistic volume of real applications, exactly the kind of volume the Placement Office would see during peak application season.

The Correct Approach

Move the query outside the loop, running it once to retrieve everything needed for every record being processed in that transaction, and then work with the results already held in memory inside the loop. This is the same principle already at work in Sprint 3's duplicate check: search once, using SOQL, before looping through the results to make individual decisions — never repeat the search itself once for every record.

> **BEGINNER ANALOGY (Supplementary Explanation)**
>
> Imagine a teacher who, to find one student's name in the class register, walks all the way back to the office to fetch the register, checks one name, walks back to the office to return it, then walks back again for the next student. Doing this two hundred times for two hundred students would be exhausting and slow. The obvious fix is to bring the whole register into the classroom once, and check every name against the one register already in hand.

Situation 2 — The Same Validation Logic Duplicated in Three Methods

The scenario: the same validation logic appears in three different methods. The discussion question is: what maintenance problems could arise, and how would you redesign the solution?

Why It Is Wrong

If a rule such as the CGPA check is written out three separate times, then a future change to that rule — for example, raising the minimum CGPA — requires finding and correctly updating all three copies. Missing

even one copy leaves the system enforcing two different rules at once, silently, which is exactly the kind of inconsistency that produced the Placement Officer's original complaints.

The Correct Approach

This is precisely why Sprint 4 introduced helper methods such as validateCGPA(): the rule is written once, inside one method, and every place that needs to check CGPA calls that one method rather than re-implementing the logic. Changing the rule then means changing exactly one method.

Situation 3 — Poor Method Names (doWork(), execute(), process())

The scenario: method names such as doWork(), execute(), and process() appear throughout the project. The discussion question is: can another developer understand their purpose without opening the implementation, and what better names would you suggest?

Why It Is Wrong

These names describe nothing about the business activity being performed. Chapter 4 was explicit about this exact failure mode: good method names tell a story, and someone reading the code should understand what the software is doing without reading every statement inside the method. A method called process() gives no such story — the reader has no choice but to open it and read every line just to find out what it does.

The Correct Approach

Rename each method after the specific business activity it performs — exactly the pattern already used throughout this handbook: submitApplication(), checkDuplicate(), validateCGPA(), saveApplication(). Each of these names alone tells a reader what happens, with no need to open the method body.

Situation 4 — A Method Containing More Than Two Hundred Lines of Code

The scenario: a method contains more than two hundred lines of code. The discussion question is: how would you divide this responsibility into smaller methods?

Why It Is Wrong

A single, enormous method almost always means multiple responsibilities have been merged into one place, in direct violation of the Single Responsibility Principle from Chapter 3. Such a method is hard to read, hard to test, and dangerous to change, since a modification intended for one part of its behaviour can easily and invisibly affect another part.

The Correct Approach

This is exactly the problem Sprint 4's "Professional Design" solves in advance: instead of one large submitApplication() method containing every check, the responsibility is divided into checkDuplicate(), four validate*() helper methods, and saveApplication(), each with one clear job. A two-hundred-line method is a signal to look for the natural helper methods hiding inside it and extract them, the same way Sprint 4 extracted the eligibility checks.

> **KEY TAKEAWAYS**
>
> SOQL inside a loop scales badly and risks hitting Salesforce's query limits — move the query outside the loop.
>
> Duplicated validation logic creates a maintenance trap where one copy can be updated and another

forgotten — give each rule one home.

Poor method names (process, execute, doWork) force a reader to open the code to understand it — name methods after business activities instead.

An oversized method is usually several responsibilities merged together — divide it into focused helper methods.

COMMON MISTAKES

Writing a query inside a loop because it "only runs a few times" during testing, without considering production volume.

Copy-pasting a validation check into a new method instead of calling the existing helper method.

Leaving a placeholder method name like process() in place after the method's real purpose becomes clear.

INTERVIEW NOTES

"Why is SOQL inside a loop considered an anti-pattern in Salesforce?" — it multiplies query usage with the number of loop iterations and risks exceeding governor limits under realistic data volumes.

"How do you avoid duplicating business logic?" — extract the rule into a single, well-named helper method and call it from everywhere it is needed.

REVISION SUMMARY

Four debugging situations: SOQL in a loop, duplicated validation logic, poor method names, oversized methods.

Each situation is solved by a principle already introduced earlier in the handbook: query outside loops, one home per rule, name after business activity, one responsibility per method.

CHAPTER 12

Interview Corner

During interviews, recruiters often evaluate how you think rather than how much syntax you remember. The five questions below are the exact questions raised in the source material's "Interview Corner" section, each expanded here with a beginner-level answer and a more complete, professional-level answer.

Source: Part IV, "Interview Corner"

Question 1

> *Why should business logic be implemented in service classes rather than inside the user interface?*

Beginner-Level Answer

Because the user interface's job is just to show things on screen and take input — it isn't the right place to decide whether an application should be accepted or rejected.

Professional-Level Answer

Placing business logic inside the UI layer violates separation of concerns: the Lightning Web Component's responsibility is to accept input and display output, not to make business decisions. Keeping logic inside a dedicated service class such as ApplicationService means the same rule can be reused wherever it is needed, tested independently of the UI, and changed without risking the interface layer at all.

Question 2

> *What advantages does separating responsibilities provide?*

Beginner-Level Answer

It makes it easier to find where something happens and to change one thing without accidentally breaking something else.

Professional-Level Answer

Separated responsibilities make software easier to understand (each component has one clear job), easier to test (each responsibility can be verified in isolation), and easier to change (a modification to one responsibility, such as an eligibility rule, cannot accidentally affect an unrelated responsibility, such as saving the record). This is the direct payoff of the Single Responsibility Principle and the three-layer architecture introduced in Chapter 3.

Question 3

> *What characteristics make a method reusable?*

Beginner-Level Answer

It does one clear thing, has a name that explains what it does, and only asks for the information it actually needs.

Professional-Level Answer

A reusable method has a single, well-defined responsibility (per the Single Responsibility Principle), a name that communicates its business purpose (submitApplication, not process), a minimal and precise parameter list (only studentId and jobId, not unrelated data), and a clear, predictable result. Methods built this way — like the validate*() helpers from Sprint 4 — can be called from multiple places with confidence, because their behaviour is easy to predict without reading their implementation.

Question 4

> *How does good naming improve software quality?*

Beginner-Level Answer

Good names let you understand what code does just by reading its name, without having to read every line inside it.

Professional-Level Answer

As Chapter 4 states directly, good method names tell a story: a reader should understand what the software is doing without reading every statement inside the method. This dramatically reduces the time other developers spend understanding existing code, reduces the chance of misuse (calling a method for the wrong purpose), and makes debugging faster, since a stack trace full of well-named methods already tells most of the story of what the program was doing when it failed.

Question 5

> *If tomorrow the Placement Office changes its eligibility policy, how should your design minimise the required code changes?*

Beginner-Level Answer

Because each eligibility rule already has its own separate method, you would only need to change the one method for the rule that changed.

Professional-Level Answer

Because Sprint 4's design places each eligibility rule inside its own helper method (validateCGPA, validateBacklogs, validateBranch, validateGraduationYear), a change to one policy — say, the minimum CGPA — is isolated entirely to validateCGPA(). No other method needs to be touched, no duplicated logic needs to be hunted down and updated (as discussed in Chapter 11's Situation 2), and the coordinating method, submitApplication(), does not need to change at all, since it merely calls the helper methods without knowing their internal details.

What Interviewers Expect

Across all five questions, the pattern an interviewer is listening for is the same one this handbook has repeated since Chapter 1: an answer that starts from the business reasoning (why does this rule or structure exist?) rather than jumping straight to syntax. A candidate who can explain why ApplicationService is separated from StudentService, without writing a single line of code, is demonstrating exactly the habit the Technical Lead describes in Chapter 3: "Do I understand the business well enough to explain my solution without mentioning code?"

> **KEY TAKEAWAYS**

Interview questions in this material test engineering judgment, not memorised syntax.

Every strong answer ties back to one of the core principles: separation of concerns, single responsibility, meaningful naming, or build-small-test-often.

The eligibility-change question is the clearest test of whether the Sprint 4 helper-method design was actually understood, not just copied.

REVISION SUMMARY

Five interview questions, each answerable at both a beginner and professional level.

The professional answers all reuse vocabulary and reasoning already built up across Chapters 1–10.

Interviewers are testing whether you understand why the design looks the way it does.

CHAPTER 13

Sprint Retrospective

Every successful Agile sprint ends with reflection. The source material allocates fifteen minutes for a pod to discuss four retrospective questions. Each is expanded below.

Source: Part IV, "Sprint Retrospective"

What Went Well?

The retrospective asks each pod to identify one design decision the team is proud of. Looking back across this sprint, several strong candidates stand out from the material itself: the decision to separate StudentService, JobService, and ApplicationService before writing any Apex (Chapter 3); the decision to break eligibility validation into small, named helper methods rather than one large block (Chapter 8); and the decision to place the DML save operation last, only after every validation had already passed (Chapter 9).

What Was Difficult?

The retrospective asks which concept required the greatest discussion, and why. Based on how much space the source material devotes to it, the strongest candidate is the question of where each responsibility belongs — the recurring "is this responsibility in the right place?" question from Chapter 3. Unlike a syntax question with one correct answer, this is a judgment call, which is exactly why the material builds in pod discussions and peer review checkpoints around it (for example, Sprint 1's Peer Review Checklist and Sprint 3's one-minute understandability check).

What Would You Improve?

The retrospective asks: if you had another two hours, what changes would you make before releasing this software? The material itself leaves two threads deliberately open rather than resolved: Sprint 4's challenge question about how the design would need to change if every company had different eligibility rules, and Sprint 5's question of exactly how save failures should be communicated in more sophisticated detail as the application matures. Both are natural candidates for "what would you improve" — not because the current design is wrong, but because the material explicitly frames them as open engineering questions rather than settled ones.

One Lesson to Remember

Each team member is asked to write one engineering lesson learned — not a programming concept, but an engineering principle. The source material gives its own list of examples:

- Understand before implementing.
- Keep responsibilities separate.
- Good names improve readability.
- Build incrementally.
- Every method should solve one problem.

The Tech Lead's Closing Note

The source material closes this sprint with a reflection worth reproducing in full, because it summarises the entire handbook's philosophy in a few sentences:

> *"Today you wrote your first business service in Apex. Months from now, you may not remember every keyword or every syntax rule. That is perfectly normal. What matters is something much more valuable. Today you learned that software is not created by typing quickly. It is created by thinking carefully. Every class exists because the business has a responsibility. Every method exists because the business performs an activity. Every variable represents information that the business cares about. Every line of code should make the software more useful to its users. Never measure yourself by the number of classes you write. Measure yourself by the number of business problems you solve. That is what professional software engineers do. That is what great Salesforce developers do."*

Real Industry Examples of Why This Matters

The retrospective's insistence on "engineering lessons, not programming concepts" mirrors how engineering teams in real companies conduct retrospectives at the end of any sprint or release: the goal is not to catalogue which Apex keywords were used, but to capture durable habits — such as "we should have discussed the design before coding" or "we should have named this method more clearly the first time" — that will keep paying off on every future sprint, long after the specific code from this sprint has been rewritten or replaced.

KEY TAKEAWAYS

A sprint retrospective captures engineering lessons, not just what code was written.

This sprint's proudest decisions: separating the three services, breaking eligibility checks into helper methods, and saving data only after every validation passed.

The Tech Lead's closing message: measure yourself by business problems solved, not lines of code written.

REVISION SUMMARY

Retrospective questions: what went well, what was difficult, what would you improve, and one lesson to remember.

The five example lessons: understand before implementing, keep responsibilities separate, good names improve readability, build incrementally, one method solves one problem.

CHAPTER 14

Complete Revision Notes

This chapter condenses every earlier chapter into short, exam-ready notes, organised in the same order as the rest of the handbook.

Ch 1 — Introduction

- Sprint 4 turns a data-storing system into a decision-making system.
- Business problem: invalid applications were being accepted because no rules were enforced.
- Core principle: understand the business first, write the code afterwards.

Ch 2 — Building Business Logic with Apex

- Business logic = the intelligence that lets software make correct decisions.
- CRUD = mechanical storage; business logic = judgment applied on top of CRUD.
- Every organisation has rules; Sprint 4 translates the Placement Office's rules into software.

Ch 3 — Thinking Like an Architect

- Architecture = organised arrangement of responsibilities, decided before coding.
- Three layers: UI (LWC) -> Business (ApplicationService) -> Database.
- Single Responsibility Principle: one component, one job.
- Three services: StudentService, JobService, ApplicationService.

Ch 4 — Discovering Apex

- public class ApplicationService { } — a class represents one business responsibility.
- Methods represent business activities; name them accordingly (submitApplication).
- Parameters = only the information the activity truly needs.
- Every method must communicate a clear result.

Ch 5 — Sprint 1

- Create ApplicationService as an empty public class before adding any methods.

Ch 6 — Sprint 2

- submitApplication(Id studentId, Id jobId) receives the request; no validation yet.

Ch 7 — Sprint 3

- Check for a duplicate application before saving, using SOQL, scoped to same student + same job.

Ch 8 — Sprint 4

- Eligibility validation split into checkDuplicate, validateCGPA, validateBacklogs, validateBranch, validateGraduationYear, saveApplication.

Ch 9 — Sprint 5

- DML (insert application;) runs last, only after every validation has passed. Failures need clear messages too.

Ch 10 — Sprint 6

- Full workflow: Receive -> Validate Duplicate -> Validate Eligibility -> Save Record -> Display Result.

Ch 11 — Debugging

- Avoid: SOQL in loops, duplicated logic, poor names, oversized methods.

Ch 12 — Interview Corner

- Five questions on service classes, separation, reusability, naming, and change-resilience.

Ch 13 — Sprint Retrospective

- Reflect on engineering lessons, not just code written; measure yourself by business problems solved.

> **REMEMBER**
>
> If you can only remember one sentence from this entire handbook, remember this: understand the business first, write the code afterwards.

CHAPTER 15

Glossary

Every important term used across this handbook, defined in the sense the source material uses it.

Business Logic — The intelligence that allows software to behave according to the needs of an organisation — the rules that let software make correct decisions rather than simply store information.

Architecture — The organised arrangement of responsibilities within a software system, decided before implementation.

Apex — Salesforce's programming language, used to implement business logic as executable software.

Class — A named container in Apex that represents one business responsibility (e.g., ApplicationService).

Method — A named unit of behaviour inside a class that represents one business activity (e.g., submitApplication).

Parameter — A piece of information supplied to a method so it can perform its responsibility (e.g., studentId, jobId).

SOQL — Salesforce Object Query Language — used to retrieve information required for decision-making, such as checking whether a duplicate application already exists.

DML — Data Manipulation Language — used to insert, update, delete, or upsert records, storing the results of business decisions.

Validation — The process of checking that a record satisfies business rules (such as CGPA, backlogs, branch, or graduation year) before it is allowed to be saved.

Service Class — An Apex class dedicated to one business responsibility, such as StudentService, JobService, or ApplicationService.

ApplicationService — The service class responsible for the application process: receiving applications, checking duplicates, validating eligibility, saving applications, and returning results.

StudentService — The service class responsible for student-related operations: registering students, updating profiles, verifying academic information, checking placement status.

JobService — The service class responsible for job-related operations: creating job postings, updating eligibility criteria, closing expired opportunities, publishing jobs.

Eligibility — Whether a student satisfies a company's requirements (CGPA, backlogs, branch, graduation year) to apply for a job.

Duplicate Validation — The check that prevents a student from applying to the same job more than once.

Responsibility — One well-defined job or concern that a component (class, method, or layer) is accountable for.

Separation of Concerns — The principle that each part of a system should own one category of responsibility and not reach into another part's job.

Single Responsibility Principle — A component should have one, and only one, primary responsibility.

Layered Architecture — Organising a system into layers (UI, Business, Database) each with a distinct responsibility.

Workflow — The complete, ordered sequence of steps a business process follows — e.g., Receive -> Validate Duplicate -> Validate Eligibility -> Save -> Display Result.

Helper Method — A small, focused method (such as validateCGPA) called by a coordinating method to keep each responsibility separate and readable.

Governor Limits — Salesforce's platform-enforced limits (such as on the number of SOQL queries per transaction) that make patterns like SOQL-inside-a-loop dangerous at scale.

CHAPTER 16

Interview Cheat Sheet

A condensed, quick-reference version of Chapter 12, for last-minute revision before an interview.

Most Important Questions and Expected Answers

Question	One-Line Expected Answer
Why put business logic in service classes, not the UI?	UI's job is input/output only; decisions belong in a dedicated, reusable, testable service class.
What does separating responsibilities give you?	Easier to understand, test, and change without side effects on unrelated parts.
What makes a method reusable?	One responsibility, a clear business-focused name, minimal parameters, a predictable result.
How does good naming improve quality?	Readers understand intent without reading the implementation — faster reviews, faster debugging.
How do you minimise change when a rule changes?	Keep each rule in its own helper method, so only that method needs to change.
Why avoid SOQL inside loops?	Query count scales with loop iterations and risks exceeding Salesforce governor limits.
Why validate before saving, not after?	An invalid record should never be written to the database in the first place.

Tips

- Always answer from the business reasoning first, then mention the technical mechanism — this mirrors exactly how the Technical Lead frames every explanation in this handbook.
- If asked to design something new, think out loud in the same order the source material teaches: business requirement -> responsibilities -> architecture -> Apex class/method -> parameters -> result.
- If you don't know a specific syntax detail, say so honestly, but demonstrate that you understand where the logic should live and why — that is what this material calls thinking like an engineer.
- When explaining ApplicationService, be ready to explain what it is NOT responsible for (student registration, job publishing) just as clearly as what it is responsible for.

> **TEAM LEAD TIP**
>
> Before any interview, ask yourself the same question the Technical Lead asks before any coding: "Do I understand the business well enough to explain my solution without mentioning code?" If yes, you are ready.

CHAPTER 17

Professional Best Practices

This closing chapter collects every best practice referenced across the handbook into one place, organised by theme.

Salesforce Best Practices

- Understand the business requirement fully before writing any Apex.
- Design responsibilities (which class does what) before implementation.
- Validate before saving — never let an invalid record reach the database.
- Never place a SOQL query inside a loop — retrieve what you need once, outside the loop.
- Perform the DML (save) operation only after all validations have passed.

Clean Code

- Business before Code.
- Architecture before Implementation.
- One Class = One Responsibility.
- One Method = One Responsibility.
- Validate before Saving.
- Keep methods small — if a method grows past a couple of hundred lines, look for helper methods hiding inside it.
- Avoid duplicate code — give every rule exactly one home.

Naming Standards

- Name classes after the business responsibility they represent (ApplicationService, StudentService, JobService).
- Name methods after the business activity they perform (submitApplication, validateCGPA, saveApplication).
- Avoid vague, technical-sounding names such as process(), execute(), or doWork().
- Name parameters after the business information they represent (studentId, jobId), not generic placeholders.

Architecture Guidelines

- Keep the UI layer limited to accepting input and displaying output.
- Keep business decisions inside dedicated service classes, not inside Lightning Web Components.
- Keep the database layer limited to storing and retrieving records.
- Ask "is this responsibility in the right place?" whenever a class or method starts doing too much.

Reusable Methods

- Give each method one clear responsibility so it can be safely reused elsewhere.
- Keep parameter lists minimal and purposeful.
- Make sure a method's name, parameters, and result are enough for another developer to use it correctly without reading its implementation.

Maintainability

- Isolate each business rule inside its own helper method, so a policy change touches only one place.
- Avoid duplicating logic across multiple methods — this was Debugging Situation 2 in Chapter 11.
- Prefer several small, clearly-named methods over one large, ambiguous one.

Scalability

- Design with realistic data volumes in mind — a pattern that works for one record can fail for two hundred (Chapter 11, Situation 1).
- Keep validation logic structured so it can be extended (for example, per-company eligibility rules) without a full rewrite.

Testing

ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)

The four source documents do not describe a specific testing methodology (such as unit test structure or code coverage targets) in detail. What they do establish clearly is the incremental philosophy that testing depends on: "build small, test often, improve continuously" (Section 4.13). The practical implication for testing is that each small, single-responsibility method — such as validateCGPA() — is naturally easier to verify in isolation than one large method would be, because its expected behaviour is narrow and clearly defined.

Documentation

ADDITIONAL EXPLANATION (Supplementary — Beyond the Source Text)

The source documents themselves are, in effect, a model of the documentation practice they teach: each sprint records its Business Requirement, the engineering discussion behind it, the coding task, and the expected outcome, before any retrospective. Applying the same habit to real project work — writing down the business requirement and the reasoning behind a design choice, not just the final code — is the natural extension of this material's own format into ongoing professional practice.

REMEMBER

"Never measure yourself by the number of classes you write. Measure yourself by the number of business problems you solve." — Tech Lead's Closing Note, Chapter 13.

Closing Note

This handbook began with a filing cabinet that could store anything but understood nothing, and ends with a fully designed, fully reasoned business service capable of receiving an application, checking it for duplicates, validating it against four separate eligibility rules, saving it only once it is known to be correct, and telling the student exactly what happened. Every decision along the way — every class, every method, every parameter — was made for a reason that could be explained in plain business language before a single line of Apex was written. That is the discipline this handbook exists to teach, and it is the discipline that separates code that merely runs from software that a business can actually depend on.

APPENDIX A

Consolidated ApplicationService Structure

The source material introduces ApplicationService gradually, one sprint at a time, and never prints one single, final version of the whole class in a single place. This appendix combines the structure described across Sprints 1 through 6 into one consolidated view, purely as a study aid. It reflects only the method names, parameters, call order, and comments already described in the source chapters — it is a structural summary, not a newly invented implementation.

```
public class ApplicationService {

    // Sprint 2: receive the request
    public void submitApplication(Id studentId, Id jobId){

        // Sprint 3: reject if a duplicate application exists
        checkDuplicate(studentId, jobId);

        // Sprint 4: reject if the student is not eligible
        validateCGPA(studentId, jobId);
        validateBacklogs(studentId, jobId);
        validateBranch(studentId, jobId);
        validateGraduationYear(studentId, jobId);

        // Sprint 5: store the record, once every check has passed
        saveApplication(studentId, jobId);

        // Section 4.12: communicate a clear result to the user
    }

    // Sprint 3
    private void checkDuplicate(Id studentId, Id jobId){ }

    // Sprint 4
    private void validateCGPA(Id studentId, Id jobId){ }
    private void validateBacklogs(Id studentId, Id jobId){ }
    private void validateBranch(Id studentId, Id jobId){ }
    private void validateGraduationYear(Id studentId, Id jobId){ }

    // Sprint 5
    private void saveApplication(Id studentId, Id jobId){ }

}
```

Note: helper method bodies and their exact SOQL/DML statements are described narratively in the source documents (search before saving, insert application; once validation passes) rather than printed as a complete final listing. This structural skeleton should be read alongside Chapters 6–10, not as a substitute for them.

APPENDIX B

Diagram Index

Every diagram used in this handbook, gathered here for quick reference.

1. The Journey of a Student Application (Chapter 3)

```
Student -> LWC -> ApplicationService -> Eligibility Validation -> Database ->
Confirmation
```

2. Sprint 4 Professional Design (Chapters 8, Appendix A)

```
submitApplication()
  |-- checkDuplicate()
  |-- validateCGPA()
  |-- validateBacklogs()
  |-- validateBranch()
  |-- validateGraduationYear()
  |-- saveApplication()
```

3. Duplicate-Check Flow (Chapter 7)

```
Receive Request -> Search Existing Record -> Duplicate? -> Reject or Continue
```

4. Complete Workflow (Chapter 10)

```
Receive Application -> Validate Duplicate -> Validate Eligibility -> Save
Record -> Display Result
```

5. Automation Order (Best Practice Reminder)

```
Understand the Business -> Design the Architecture -> Write the Apex -> Test
Incrementally
```